SYSTEM REFERENCE DOCUMENT

Technical Documentation

Batlounge Fruits Distribution Management System

A structured technical overview of the application's architecture, frontend and backend modules, database relationships, business rules, security controls, transaction handling, and scope boundaries.

Application	Batlounge Fruits Distribution Management System
Frontend	React single-page application with Tailwind CSS
Backend	Node.js and Express REST API
Database	MySQL via a mysql2 promise connection pool
Document	Technical documentation and system reference

Purpose-built reference for developers, maintainers, reviewers, and project evaluators.

Document overview

This document explains how the system is organised and the key rules that guide its data handling and operational behaviour.

Section	Topic
01	Purpose and system overview
02	Architecture
03	Frontend modules
04	Backend modules
05	Database design and relationships
06	Business rules
07	Security
08	Transaction handling
09	Scope limits and reference diagrams

Implementation note: This guide documents the system description supplied for the project. For exact endpoint paths, column names, constraints, and configuration details, consult the current source code, API documentation, database schema, and the referenced diagram files.

01 Purpose and system overview

The Batlounge system digitises fruit purchasing, stock control, vendor registration, order processing, and reporting. It connects a React single-page application to an Express REST API backed by a MySQL database.

The system is intended to centralise operational records, apply consistent business rules, and help authorised users manage the movement of products from purchasing and stock intake through to vendor orders and reporting.

02 Architecture

The application uses a three-tier architecture. Each tier has a distinct responsibility and communicates through defined interfaces.

Tier	Primary responsibilities	Technology / components
Presentation	Renders screens, captures user input, presents validation feedback, and provides responsive navigation.	React pages, reusable Tailwind components, authentication context, Axios services.
Application	Exposes REST endpoints, validates requests, applies business rules, authenticates users, and enforces role permissions.	Node.js, Express routes and controllers, middleware, validation utilities.
Data	Persists users, catalogue records, purchasing, stock-related records, vendors, orders, and order items.	MySQL tables accessed using a mysql2 promise connection pool.

Request flow

- **1. User interaction:** a user opens a page and submits an action in the React application.
- **2. API request:** the frontend service sends an HTTP request to the Express REST API, attaching a Bearer token when required.
- **3. Middleware:** authentication and role checks run before protected controller logic.
- **4. Business logic:** the controller validates the request and applies the relevant operational rules.
- **5. Database operation:** parameterised SQL queries read or change records; sensitive multi-step operations use transactions.
- **6. Response:** the API returns a response that the frontend uses to update the interface or display an error.

Architecture reference: `diagrams/ARCHITECTURE.md`

03 Frontend modules

The frontend is organised by page, shared interface component, application context, service wrapper, and route protection.

Directory / module	Responsibility
`pages/`	Login, Dashboard, Fruits, BoxSizes, Suppliers, Stock, Vendors, Orders, CreateOrder, OrderDetails, Reports, Unauthorized, and NotFound.
`components/layout/`	Shared responsive sidebar, topbar, and route layout.
`components/ui/`	Reusable buttons, inputs, selects, cards, tables, modals, badges, loaders, and empty states.
`context/`	Persisted authentication state and temporary toast notifications.
`services/`	Axios configuration and endpoint wrappers, keeping API calls reusable and consistent.
`routes/`	Role-aware route guards that control which frontend pages a user may access.

Frontend route guards improve navigation and user experience, but they are not a replacement for backend authorisation. The API must independently enforce access rules on every protected operation.

04 Backend modules

The backend separates HTTP routing, business operations, cross-cutting middleware, validation utilities, and database connectivity.

Directory / file	Responsibility
`controllers/`	Database operations and business rules for the application's resources.
`routes/`	Maps API endpoints to the appropriate controller functions and middleware.
`middleware/`	JWT authentication, role authorisation, and centralised error handling.
`utils/`	Reusable validation helpers and async error forwarding.
`config/db.js`	Creates and exports the MySQL connection pool used by the application.

This separation keeps controllers focused, makes permission checks easier to review, and helps keep error handling and data access consistent across endpoints.

05 Database design and relationships

The database contains users, suppliers, fruits, box sizes, products, street vendors, purchases, orders, and order items.

A product represents the unique combination of one fruit and one box size. This model allows the system to track separate product variants and apply different prices and stock quantities—for example, Apples Small, Apples Medium, and Apples Large.

Relationship summary

Parent entity	Relationship	Child entity	Meaning
Supplier	One-to-many	Purchases	A supplier may be associated with multiple purchase records.
Fruit	One-to-many	Products	A fruit can be offered in multiple box-size variants.
Box size	One-to-many	Products	A box size can be used by multiple fruit variants.
Product	One-to-many	Purchases	A product may appear in multiple purchase records.
Street vendor	One-to-many	Orders	A vendor may place multiple orders.
Order	One-to-many	Order items	An order contains one or more line items.
Product	One-to-many	Order items	A product may appear in many order line items.

Database integrity

Foreign keys preserve relationships between records. The application also applies business validation before database changes, while database constraints provide an additional layer of protection against invalid references.

Entity relationship diagram reference: `diagrams/ERD.md`

06 Business rules

The following rules define valid catalogue data, purchasing and ordering behaviour, and stock integrity.

Rule area	Requirement
Catalogue	Fruit names are limited to the five required fruits. Box sizes are Small, Medium, and Large.
Supplier location	A supplier's province must be one of the nine South African provinces.
Positive values	Product prices and required quantities must be greater than zero.
Vendor eligibility	An order must belong to an existing active vendor and contain at least one item.
Stock availability	A requested quantity cannot exceed the available stock.
Authoritative pricing	Prices and order totals are calculated from database prices rather than trusted client-submitted totals.
Completion	Completing an order deducts stock inside a database transaction.
Cancellation	Cancelled orders do not deduct stock.
Finalised orders	Completed orders cannot be reopened or deleted.
Referential integrity	Foreign keys preserve relationships between related records.

These rules should be enforced on the server even when equivalent client-side validation is present. Client validation improves usability; server validation protects data integrity.

07 Security

Security controls protect credentials, API access, role-restricted operations, database queries, and error responses.

Control	Implementation
Password storage	Passwords are hashed using bcryptjs; plaintext passwords should not be stored.
Token issuance	Login returns a signed JSON Web Token (JWT) containing user ID, username, and role.
Protected requests	Protected API requests require a Bearer token in the Authorization header.
Role enforcement	Admin-only routes are enforced by backend middleware, independently of frontend route guards.
SQL injection resistance	SQL statements use parameter placeholders instead of concatenating untrusted values into SQL.
Error disclosure	Production error responses do not expose stack traces.

Security maintenance reminders

- Keep signing secrets, database credentials, and environment-specific values out of source control.
- Validate and authorise every protected API request on the server.
- Return clear, appropriate client-facing errors without exposing internal implementation details.
- Review the Admin/Sales access matrix whenever routes or business features change.

08 Transaction handling

Transactions keep related database changes consistent when an operation spans multiple records or depends on current stock.

Stock intake, order creation, order editing, and order completion use database transactions. For order creation and completion, the application locks the relevant product rows with `FOR UPDATE` before checking or changing stock.

Typical transaction lifecycle

Step	Expected behaviour
Acquire connection	Obtain a connection from the MySQL pool.
Begin transaction	Start a transaction before performing dependent changes.
Lock and validate	Lock relevant product rows where required, then verify current stock and business constraints.
Apply changes	Write the purchase/order records and update stock as part of the same transaction.
Commit or roll back	Commit only when all required operations succeed; otherwise roll back to avoid partial changes.
Release connection	Release the connection back to the pool regardless of success or failure.

Why row locking matters: locking the product rows helps prevent concurrent requests from both relying on the same stale stock quantity. The stock check and stock update should occur within the same transaction.

Implementation note: use structured error handling so that rollback is attempted on failure and the acquired connection is always released.

09 Scope limits and reference diagrams

The following boundaries describe what is and is not included in the current application scope.

- The application is single-tenant.
- Currency is displayed in Maloti (M).
- Payment processing is not included.
- SMS and email notifications are not included.
- A native mobile client is not included.

Related project references

Reference	Purpose
`diagrams/ARCHITECTURE.md`	Architecture diagram and component-level overview.
`diagrams/ERD.md`	Entity relationship diagram for the database.

End of technical documentation

Keep this document aligned with the implementation. Update it when architecture, modules, database relationships, business rules, security controls, transaction behaviour, or scope changes.